

SKILL-11

Session-21:

Task-1: To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor 4 Components

```
GNU nano 7.2 modules.h *
#ifndef MODULES_H
#define MODULES_H

int get_input(int *value);
int process_data(int input, int *result);
void display_result(int input, int result);

#endif
```

```
GNU nano 7.2 input.c *
#include <stdio.h>
#include "modules.h"

int get_input(int *value)
{
    if (value == NULL)
    {
        fprintf(stderr, "[Input] Error: NULL pointer.\n");
        return -1;
    }

    printf("[Input] Enter a positive integer: ");

    if (scanf("%d", value) != 1)
    {
        fprintf(stderr, "[Input] Error: Invalid input.\n");
        return -1;
    }

    if (*value <= 0)
    {
        fprintf(stderr, "[Input] Error: Value must be positive.\n");
        return -1;
    }

    printf("[Input] Input accepted: %d\n", *value);
    return 0;
}
```

```
hasini0404@hasini: ~/Operati + v
GNU nano 7.2 process.c *
#include <stdio.h>
#include "modules.h"

int process_data(int input, int *result)
{
    if (result == NULL)
    {
        fprintf(stderr, "[Process] Error: NULL result pointer.\n");
        return -1;
    }

    if (input <= 0)
    {
        fprintf(stderr, "[Process] Error: Invalid input.\n");
        return -1;
    }

    /* Processing operation */
    *result = input * input;

    printf("[Process] Processing completed.\n");
    printf("[Process] %d x %d = %d\n",
        input, input, *result);

    return 0;
}
```

```
GNU nano 7.2 output.c *
#include <stdio.h>
#include "modules.h"

void display_result(int input, int result)
{
    printf("\n===== \n");
    printf("          FINAL RESULT\n");
    printf("===== \n");

    printf("Input      : %d\n", input);
    printf("Result     : %d\n", result);

    printf("Status      : SUCCESS\n");

    printf("===== \n");
}
```

```
GNU nano 7.2                                main.c *
#include <stdio.h>
#include "modules.h"

int main()
{
    int input;
    int result;

    printf("=====\n");
    printf("          MODULAR SYSTEM INTEGRATION\n");
    printf("=====\n");

    printf("\n[Main] Starting end-to-end execution...\n");

    /* Module 1: Input */
    if (get_input(&input) != 0)
    {
        fprintf(stderr, "[Main] Input module failed.\n");
        return 1;
    }

    /* Module 2: Processing */
    if (process_data(input, &result) != 0)
    {
        fprintf(stderr, "[Main] Processing module failed.\n");
        return 1;
    }

    /* Module 3: Output */
    display_result(input, result);

    printf("\n[Main] All modules executed successfully.\n");
    printf("[Main] End-to-end execution completed.\n");

    return 0;
}
```

  Help  Write Out  Where Is  Cut  Execute

```
hasini0404@hasini: ~/Operating_Systems$ mkdir -p skill11s21task1
hasini0404@hasini: ~/Operating_Systems$ cd skill11s21task1
hasini0404@hasini: ~/Operating_Systems/skill11s21task1$ nano modules.h
hasini0404@hasini: ~/Operating_Systems/skill11s21task1$ nano input.c
hasini0404@hasini: ~/Operating_Systems/skill11s21task1$ nano process.c
hasini0404@hasini: ~/Operating_Systems/skill11s21task1$ nano output.c
hasini0404@hasini: ~/Operating_Systems/skill11s21task1$ nano main.c
hasini0404@hasini: ~/Operating_Systems/skill11s21task1$ ls
input.c  main.c  modules.h  output.c  process.c
hasini0404@hasini: ~/Operating_Systems/skill11s21task1$ cat modules.h
#ifndef MODULES_H
#define MODULES_H

int get_input(int *value);
int process_data(int input, int *result);
void display_result(int input, int result);

#endif
hasini0404@hasini: ~/Operating_Systems/skill11s21task1$ cat input.c
#include <stdio.h>
#include "modules.h"

int get_input(int *value)
{
    if (value == NULL)
    {
        fprintf(stderr, "[Input] Error: NULL pointer.\n");
        return -1;
    }

    printf("[Input] Enter a positive integer: ");

    if (scanf("%d", value) != 1)
    {
        fprintf(stderr, "[Input] Error: Invalid input.\n");
        return -1;
    }

    if (*value <= 0)
    {
        fprintf(stderr, "[Input] Error: Value must be positive.\n");
```

```

        return -1;
    }

    if (input <= 0)
    {
        fprintf(stderr, "[Process] Error: Invalid input.\n");
        return -1;
    }

    /* Processing operation */
    *result = input * input;

    printf("[Process] Processing completed.\n");
    printf("[Process] %d x %d = %d\n",
           input, input, *result);

    return 0;
}
hasini0404@hasini: ~/Operating_Systems/skill11s21task1$ cat output.c
cat output.c: command not found
hasini0404@hasini: ~/Operating_Systems/skill11s21task1$ cat output.c
#include <stdio.h>
#include "modules.h"

void display_result(int input, int result)
{
    printf("\n=====\\n");
    printf("          FINAL RESULT\\n");
    printf("=====\\n");

    printf("Input      : %d\\n", input);
    printf("Result     : %d\\n", result);

    printf("Status      : SUCCESS\\n");

    printf("=====\\n");
}
hasini0404@hasini: ~/Operating_Systems/skill11s21task1$ cat main.c
#include <stdio.h>
#include "modules.h"

```

```

int main()
{
    int input;
    int result;

    printf("=====\\n");
    printf("          MODULAR SYSTEM INTEGRATION\\n");
    printf("=====\\n");

    printf("\\n[Main] Starting end-to-end execution...\\n");

    /* Module 1: Input */
    if (get_input(&input) != 0)
    {
        fprintf(stderr, "[Main] Input module failed.\\n");
        return 1;
    }

    /* Module 2: Processing */
    if (process_data(input, &result) != 0)
    {
        fprintf(stderr, "[Main] Processing module failed.\\n");
        return 1;
    }

    /* Module 3: Output */
    display_result(input, result);

    printf("\\n[Main] All modules executed successfully.\\n");
    printf("[Main] End-to-end execution completed.\\n");

    return 0;
}
hasini0404@hasini: ~/Operating_Systems/skill11s21task1$ gcc -Wall -Wextra main.c input.c process.c output.c -o skill11s21task1
hasini0404@hasini: ~/Operating_Systems/skill11s21task1$ ./skill11s21task1
=====
          MODULAR SYSTEM INTEGRATION
=====
[Main] Starting end-to-end execution...

```

```

hasini0404@hasini:~/Operating_Systems/skill11s21task1$ gcc -Wall -Wextra main
.c input.c process.c output.c -o skill11s21task1
hasini0404@hasini:~/Operating_Systems/skill11s21task1$ ./skill11s21task1
=====
MODULAR SYSTEM INTEGRATION
=====

[Main] Starting end-to-end execution...
[Input] Enter a positive integer: 5
[Input] Input accepted: 5
[Process] Processing completed.
[Process] 5 x 5 = 25

=====
FINAL RESULT
=====
Input      : 5
Result     : 25
Status     : SUCCESS
=====

[Main] All modules executed successfully.
[Main] End-to-end execution completed.
hasini0404@hasini:~/Operating_Systems/skill11s21task1$ |

```

Observation

The program was successfully divided into four independent components: input, processing, output, and main controller modules. A common header file was used to define and validate the interfaces between the modules. The modules were compiled separately and linked together to perform end-to-end execution. Error handling was implemented for invalid input, non-positive values, and invalid pointers, improving the reliability of the system. The final execution successfully passed data from the input module through the processing module to the output module. Thus, module integration, interface validation, end-to-end testing, error handling, reliability improvement, and four-component refactoring were successfully demonstrated.

Task-2: TO Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX_INPUT 100

void log_error(const char *message)
{
    FILE *file = fopen("error.log", "a");

    if (file == NULL)
    {
        printf("[LOG ERROR] Unable to open error.log\n");
        return;
    }

    time_t now = time(NULL);
    struct tm *t = localtime(&now);

    fprintf(file, "[%04d-%02d-%02d %02d:%02d:%02d] %s\n",
            t->tm_year + 1900,
            t->tm_mon + 1,
            t->tm_mday,
            t->tm_hour,
            t->tm_min,
            t->tm_sec,
            message);

    fclose(file);
}

void show_error(const char *message)
{
    printf("[ERROR] %s\n", message);
    log_error(message);
}

```

```

int is_valid_number(const char *str)
{
    char *end;
    strtol(str, &end, 10);

    return (*str != '\0' && *end == '\0');
}

void process_command(char *input)
{
    char command[20];
    char arg1[20];
    char arg2[20];

    int count = sscanf(input, "%19s %19s %19s", command, arg1, arg2);

    /* Handle empty input */
    if (count == 0)
    {
        show_error("Empty command entered.");
        return;
    }

    /* ADD command */
    if (strcmp(command, "ADD") == 0)
    {
        if (count != 3)
        {
            show_error("Invalid syntax. Use: ADD <number> <number>");
            return;
        }

        if (!is_valid_number(arg1) || !is_valid_number(arg2))
        {
            show_error("Invalid number in ADD command.");
        }
    }
}

```

```

}

int main()
{
    char input[MAX_INPUT];

    printf("=====\n");
    printf("      RUNTIME ERROR HANDLING SYSTEM\n");
    printf("=====\n");

    printf("\nSupported commands:\n");
    printf("  ADD <number> <number>\n");
    printf("  DIV <number> <number>\n");
    printf("  EXIT\n");

    printf("\nEnter commands to test error handling.\n");

    while (1)
    {
        printf("\nCommand> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            show_error("Input reading failed.");
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        process_command(input);
    }

    printf("\n[SYSTEM] Program terminated gracefully.\n");

    return 0;
}

```

```

hasini0404@hasini: ~/Operating_Systems/skill11s21task1/skill11s22task1$ ./skill
11s22task1
=====
      RUNTIME ERROR HANDLING SYSTEM
=====

Supported commands:
  ADD <number> <number>
  DIV <number> <number>
  EXIT

Enter commands to test error handling.

Command> ADD 10 20
[SUCCESS] 10 + 20 = 30

Command> DIV 20 / 5
[ERROR] Invalid number in DIV command.

Command> DIV 20 5
[SUCCESS] 20 / 5 = 4

Command> ^Z
[41+ Stopped ./skill11s22task1
hasini0404@hasini: ~/Operating_Systems/skill11s21task1/skill11s22task1$ ./skill
11s22task1
=====
      RUNTIME ERROR HANDLING SYSTEM
=====

Supported commands:
  ADD <number> <number>
  DIV <number> <number>
  EXIT

Enter commands to test error handling.

Command> DIV 20 0
[ERROR] Runtime error: Division by zero.

Command> |

```

```

[4]+ Stopped ./skill1s22task1
hasini0404@hasini: ~/Operating_Systems/skill1s21task1/skill1s22task1$ ./skill1s22task1
=====
          RUNTIME ERROR HANDLING SYSTEM
=====

Supported commands:
  ADD <number> <number>
  DIV <number> <number>
  EXIT

Enter commands to test error handling.

Command> DIV 20 0
[ERROR] Runtime error: Division by zero.

Command> ADD ten 20
[ERROR] Invalid number in ADD command.

Command> DIV 20
[ERROR] Invalid syntax. Use: DIV <number> <number>

Command> MULTIPLY 10 20
[ERROR] Unknown command.

```

Observation

The runtime error handling system was successfully implemented to detect and manage different types of errors during program execution. The program generated clear error messages for invalid syntax, invalid numerical input, unknown commands, and the runtime error of division by zero. Detected failures were recorded in an error.log file with timestamps for later analysis. Instead of terminating when an error occurred, the program handled the error and returned to the command prompt, demonstrating graceful recovery. Valid commands were processed successfully after error conditions, confirming reliable execution. Thus, runtime error detection, message generation, failure logging, invalid syntax handling, graceful recovery, and testing of multiple error scenarios were successfully demonstrated.

SESSION-22:

Task-1:

To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define SIZE 5

int main()
{
    int *numbers;
    char *message;

    printf("=====\n");
    printf("          VALGRIND MEMORY TEST\n");
    printf("=====\n");

    /* Allocate memory for integer array */
    numbers = malloc(SIZE * sizeof(int));

    if (numbers == NULL)
    {
        printf("Error: Memory allocation failed for numbers.\n");
        return 1;
    }

    /* Allocate memory for message */
    message = malloc(50 * sizeof(char));

    if (message == NULL)
    {
        printf("Error: Memory allocation failed for message.\n");
        free(numbers);
        return 1;
    }

    /* Initialize allocated memory */
    for (int i = 0; i < SIZE; i++)
    {

```

```

GNU nano 7.2                                skill1s21task2.c *
        free(numbers);
        return 1;
    }

    /* Initialize allocated memory */
    for (int i = 0; i < SIZE; i++)
    {
        numbers[i] = (i + 1) * 10;
    }

    strcpy(message, "Memory allocation successful.");
    printf("\nAllocated integer array:\n");
    for (int i = 0; i < SIZE; i++)
    {
        printf("numbers[%d] = %d\n", i, numbers[i]);
    }

    printf("\nMessage: %s\n", message);

    /*
     * Release all dynamically allocated resources.
     */
    free(numbers);
    free(message);

    /* Prevent dangling pointers */
    numbers = NULL;
    message = NULL;

    printf("\nMemory released successfully.\n");
    printf("Program cleanup completed.\n");

    return 0;
}

```

```

hasini0404@hasini: ~/Operating_Systems$ cd skill11s21task2
hasini0404@hasini: ~/Operating_Systems/skill11s21task2$ nano skill11s21task2.c
hasini0404@hasini: ~/Operating_Systems/skill11s21task2$ gcc -Wall -Wextra -g s
kill11s21task2.c -o skill11s21task2
hasini0404@hasini: ~/Operating_Systems/skill11s21task2$ ./skill11s21task2
=====
      VALGRIND MEMORY TEST
=====

Allocated integer array:
numbers[0] = 10
numbers[1] = 20
numbers[2] = 30
numbers[3] = 40
numbers[4] = 50

Message: Memory allocation successful.

Memory released successfully.
Program cleanup completed.
hasini0404@hasini: ~/Operating_Systems/skill11s21task2$ valgrind --version
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind # version 3.26.0, or
sudo apt install valgrind # version 1:3.22.0-0ubuntu2
See 'snap info valgrind' for additional versions.
hasini0404@hasini: ~/Operating_Systems/skill11s21task2$ sudo apt update
[sudo] password for ruhi0404:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
59 packages can be upgraded. Run 'apt list --upgradable' to see them.
hasini0404@hasini: ~/Operating_Systems/skill11s21task2$ sudo apt install valgr
ind
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:

```

```

Setting up libc6-dbg:amd64 (2.39-0ubuntu8.9) ...
Setting up libipt2 (2.0.6-1build1) ...
Setting up libbabeltrace1:amd64 (1.5.11-3build3) ...
Setting up valgrind (1:3.22.0-0ubuntu3) ...
Setting up libdebuginfod1t64:amd64 (0.190-1.1ubuntu0.1) ...
Setting up libsource-highlight4t64:amd64 (3.1.9-4.3build1) ...
Setting up gdb (15.1-1ubuntu1~24.04.1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.9) ...
hasini0404@hasini: ~/Operating_Systems/skill11s21task2$ valgrind --leak-check=
full --show-leak-kinds=all ./skill11s21task2
==5384== Memcheck, a memory error detector
==5384== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==5384== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==5384== Command: ./skill11s21task2
==5384==
=====
VALGRIND MEMORY TEST
=====

Allocated integer array:
numbers[0] = 10
numbers[1] = 20
numbers[2] = 30
numbers[3] = 40
numbers[4] = 50

Message: Memory allocation successful.

Memory released successfully.
Program cleanup completed.
==5384==
==5384== HEAP SUMMARY:
==5384==    in use at exit: 0 bytes in 0 blocks
==5384==    total heap usage: 3 allocs, 3 frees, 1,094 bytes allocated
==5384==
==5384== All heap blocks were freed -- no leaks are possible
==5384==
==5384== For lists of detected and suppressed errors, rerun with: -s
==5384== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
hasini0404@hasini: ~/Operating_Systems/skill11s21task2$ |

```

Observation

The program was successfully tested using Valgrind to analyze dynamic memory allocation and cleanup. Memory was allocated dynamically for an integer array and a character buffer using `malloc()`. A temporary leak test was performed by removing the corresponding `free()` statements, and Valgrind successfully identified the unreleased memory as a memory leak. The error was corrected by releasing all dynamically allocated resources using `free()`. The corrected program was executed again with Valgrind, which confirmed that all heap blocks were freed and no memory leaks or errors remained. Thus, Valgrind execution, memory leak identification, error correction, resource release, cleanup verification, and documentation of findings were successfully demonstrated.

Task-2: To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports

```
#include <stdio.h>
#include <string.h>

int main()
{
    char operation[10];
    int a, b;

    printf("Enter operation and two numbers: ");

    if (scanf("%9s %d %d", operation, &a, &b) != 3)
    {
        printf("ERROR: Invalid input\n");
        return 1;
    }

    if (strcmp(operation, "ADD") == 0)
    {
        printf("RESULT: %d\n", a + b);
    }
    else if (strcmp(operation, "SUB") == 0)
    {
        printf("RESULT: %d\n", a - b);
    }
    else if (strcmp(operation, "MUL") == 0)
    {
        printf("RESULT: %d\n", a * b);
    }
    else if (strcmp(operation, "DIV") == 0)
    {
        if (b == 0)
        {
            printf("ERROR: Division by zero\n");
            return 1;
        }

        printf("RESULT: %d\n", a / b);
    }
}
```

```

else if (strcmp(operation, "Div") == 0)
{
    if (b == 0)
    {
        printf("ERROR: Division by zero\n");
        return 1;
    }

    printf("RESULT: %d\n", a / b);
}
else
{
    printf("ERROR: Unknown operation\n");
    return 1;
}

return 0;
}

```

⌘G Help
⌘O Write Out
⌘W Where Is
⌘K Cut
⌘T Execute

```

#!/bin/bash

PROGRAM="./calculator"
REPORT="test_report.txt"

TOTAL=0
PASSED=0
FAILED=0

echo "===== > \"$REPORT\""
echo "      AUTOMATED TEST REPORT" >> "$REPORT"
echo "===== >> \"$REPORT\""
echo "" >> "$REPORT"

run_test()
{
    TEST_NAME="$1"
    INPUT="$2"
    EXPECTED="$3"

    TOTAL=$((TOTAL + 1))

    ACTUAL=$(echo "$INPUT" | $PROGRAM 2>&1)

    if [[ "$ACTUAL" == *"$EXPECTED"* ]]
    then
        echo "[PASS] $TEST_NAME"
        echo "[PASS] $TEST_NAME" >> "$REPORT"
        echo "  Expected : $EXPECTED" >> "$REPORT"
        echo "  Actual   : $ACTUAL" >> "$REPORT"
        PASSED=$((PASSED + 1))
    else
        echo "[FAIL] $TEST_NAME"
        echo "[FAIL] $TEST_NAME" >> "$REPORT"
        echo "  Expected : $EXPECTED" >> "$REPORT"
        echo "  Actual   : $ACTUAL" >> "$REPORT"
        FAILED=$((FAILED + 1))
    fi
}

```

```

"RESULT: 42"

run_test "Division Test" \
"DIV 20 5" \
"RESULT: 4"

run_test "Division By Zero Test" \
"DIV 20 0" \
"ERROR: Division by zero"

run_test "Unknown Operation Test" \
"XYZ 10 20" \
"ERROR: Unknown operation"

run_test "Invalid Input Test" \
"HELLO" \
"ERROR: Invalid input"

echo "======"
echo "TEST SUMMARY"
echo "======"
echo "Total Tests : $TOTAL"
echo "Passed Tests : $PASSED"
echo "Failed Tests : $FAILED"
echo "======"

echo "======" >> "$REPORT"
echo "TEST SUMMARY" >> "$REPORT"
echo "======" >> "$REPORT"
echo "Total Tests : $TOTAL" >> "$REPORT"
echo "Passed Tests : $PASSED" >> "$REPORT"
echo "Failed Tests : $FAILED" >> "$REPORT"
echo "======" >> "$REPORT"

echo ""
echo "Report generated: $REPORT"

```

```

~$ cd ~/Operating_Systems
hasini0404@hasini: ~/Operating_Systems$ mkdir -p skill11s22task2
hasini0404@hasini: ~/Operating_Systems$ cd skill11s22task2
hasini0404@hasini: ~/Operating_Systems/skill11s22task2$ nano calculator.c
hasini0404@hasini: ~/Operating_Systems/skill11s22task2$ gcc -Wall -Wextra calculator.c -o calculator
hasini0404@hasini: ~/Operating_Systems/skill11s22task2$ ./calculator
Enter operation and two numbers: ADD 10 20
RESULT: 30
hasini0404@hasini: ~/Operating_Systems/skill11s22task2$ ./calculator
Enter operation and two numbers: DIV 20 5
RESULT: 4
hasini0404@hasini: ~/Operating_Systems/skill11s22task2$ ./calculator
Enter operation and two numbers: Div
^Z
[1]+  Stopped                  ./calculator
hasini0404@hasini: ~/Operating_Systems/skill11s22task2$
hasini0404@hasini: ~/Operating_Systems/skill11s22task2$ nano test_script.sh
hasini0404@hasini: ~/Operating_Systems/skill11s22task2$ chmod +x test_script.sh
hasini0404@hasini: ~/Operating_Systems/skill11s22task2$ ls -l
-rw-r--r-- 1 hasini0404 hasini 1024 Nov 14 14:14 calculator.c
-rwxr-xr-x 1 hasini0404 hasini 1024 Nov 14 14:14 test_script.sh
hasini0404@hasini: ~/Operating_Systems/skill11s22task2$ ./test_script.sh
Starting automated tests...

[PASS] Addition Test
[PASS] Subtraction Test
[PASS] Multiplication Test
[PASS] Division Test
[PASS] Division By Zero Test
[PASS] Unknown Operation Test
[PASS] Invalid Input Test
=====
TEST SUMMARY
=====
Total Tests : 7
Passed Tests : 7

```

```

Failed Tests : 0
=====

Report generated: test_report.txt
hasini0404@hasini: ~/Operating_Systems/skill11s22task2$ cat test_report.txt
=====
AUTOMATED TEST REPORT
=====

[PASS] Addition Test
Expected : RESULT: 30
Actual   : Enter operation and two numbers: RESULT: 30

[PASS] Subtraction Test
Expected : RESULT: 15
Actual   : Enter operation and two numbers: RESULT: 15

[PASS] Multiplication Test
Expected : RESULT: 42
Actual   : Enter operation and two numbers: RESULT: 42

[PASS] Division Test
Expected : RESULT: 4
Actual   : Enter operation and two numbers: RESULT: 4

[PASS] Division By Zero Test
Expected : ERROR: Division by zero
Actual   : Enter operation and two numbers: ERROR: Division by zero

[PASS] Unknown Operation Test
Expected : ERROR: Unknown operation
Actual   : Enter operation and two numbers: ERROR: Unknown operation

[PASS] Invalid Input Test
Expected : ERROR: Invalid input
Actual   : Enter operation and two numbers: ERROR: Invalid input

=====
TEST SUMMARY
=====
Total Tests : 7

```

```

=====
TEST SUMMARY
=====
Total Tests : 7
Passed Tests : 7
Failed Tests : 0
=====
hasini0404@hasini: ~/Operating_Systems/skill11s22task2$ ls -l
calculator
calculator.c
test_report.txt
test_script.sh
hasini0404@hasini: ~/Operating_Systems/skill11s22task2$ |

```

Observation:

The automated testing system was successfully designed and implemented using a C program and a Bash test script. Multiple test cases were created to verify valid operations such as addition, subtraction, multiplication, and division, along with error scenarios such as division by zero, unknown operations, and invalid input. The Bash script automatically executed all test cases and compared the actual program output with the expected results. Each test case was classified as PASS or FAIL based on the comparison. A detailed test report containing individual test results and the overall test summary was generated in `test_report.txt`. Thus, test case design, test script creation, automated execution, output verification, result comparison, and report generation were successfully demonstrated.